Ders 13 Genişletilmiş Notu: Proje Test Sunumları I: Kütüphane Sistemi, Mikroservis Platformu ve Bilet Sistemi

Ders 13 Genişletilmiş Notu: Proje Test Sunumları I: Kütüphane Sistemi, Mikroservis Platformu ve Bilet Sistemi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Düzeni: Gönüllü Grupların Test Sunumları
- 2. Birinci Grup: Kütüphane Yönetim Sistemi
- 3. Hocanın Birinci Gruba Değerlendirmesi ve Projede Beklenen Minimum Koşullar
- 4. İkinci Grup: Mikroservis Mimarili Dil Eşleştirme Platformu
- 5. Üçüncü Grup: Otobüs Bilet ve Yolculuk Yönetim Sistemi
- 6. Son Hafta Sunumları İçin Yönlendirmeler
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Düzeni: Gönüllü Grupların Test Sunumları

Bu derste yeni bir konu anlatılmadı. Dönem boyunca yürütülen projenin son aşaması olan test çalışmaları, gönüllü üç grup tarafından sunuldu. Grupların büyük çoğunluğu raporlarını sisteme yüklemişti. Hoca, bu hafta yalnız istekli grupların sunacağını, kalan bütün grupların bir sonraki (son) haftada sunacağını söyledi. Sunuma başlamadan önce gruplara hazırlanmaları için yaklaşık beş dakika verdi.

Üç sunum da benzer bir iskeleti izledi:

- Test edilen yazılımın kısa tanıtımı (gerekirse daha önce yazılmış gereksinim belgesi, SRS (Software Requirements Specification) üzerinden).
- Test planı: strateji, test edilecek öğeler, kriterler, hata raporlama, sorumluluklar, takvim, riskler, ortam ve araçlar.
- Test seviyelerine göre örnekler: birim testi (unit test), entegrasyon testi (integration test), sistem testi (system test) ve kullanıcı testi (user test).

Bu nedenle ders, dönem içinde teorik olarak görülen test planı ve test sonuç raporu kavramlarının gerçek projelerde nasıl uygulandığını gösteren bir örnekler dersi niteliğindeydi.

2. Birinci Grup: Kütüphane Yönetim Sistemi

2.1 Yazılımın tanıtımı

Birinci grubun seçtiği yazılım, kütüphanelerde takibi kolaylaştırmak için web platformu üzerinde çalışan bir kütüphane yönetim sistemidir. Sistemin temel işleri kitap ödünç verme, ödünç işlemlerini kontrol etme, kitap arama, kayıtlara erişme ve rapor oluşturmadır. Grup, daha

önce hazırladığı SRS belgesinde bu sistemin fonksiyonel gereksinimlerini, kullanım senaryolarını ve fonksiyonel olmayan gereksinimlerini tanımlamıştı. Test çalışması bu belgenin üzerine kuruldu.

2.2 Test planının içeriği

Grup test planının başlıklarını şöyle sıraladı:

- Test stratejisi ve test edilecek öğeler
- Doğrulama yöntemleri
- Testlerin tamamlanma kriterleri
- Test edilebilecek öğeler
- Hata raporlama
- Test sonuç raporlama
- Test sorumlulukları ve eğitim ihtiyacı
- Test takvimi
- Risk yönetimi
- Test ortamı ve araçları

Test stratejisi ve kapsam. Planda üç tür test öngörüldü: birim testi, sistem testi ve kullanıcı testi. Bu testler beş modül üzerinde yapılacaktı: üye, kitap, yazar, yayıncı ve kütüphane ayarları modülleri. Test edilebilecek öğeler arasında ise bunlara ek olarak kullanıcı, kitap, yazar, yayıncı ve üye yönetimi ile arama modülleri, üye görüntüleme, kitap ödünç verme, ödünç kayıtlarını arama, iade süresini uzatma ve kitap iade alma gibi işlevler sayıldı.

Doğrulama yöntemi. Doğrulama iki aşamada anlatıldı. Test aşamasında, test planı doğrultusunda oluşturulan senaryolar üzerinde test işlemi yapılır. Analiz aşamasında ise test sonuçları değerlendirilir: olması gereken durum ve değerler ile sistemin yanıtı karşılaştırılır ve sonuçlar raporlanır.

Tamamlanma kriterleri. Testlerin tamamlanmış sayılması için iki koşul konuldu: Birim, sistem ve kullanıcı testlerinin tamamen başarıyla bitmiş olması ve bu testler sonucunda ortaya çıkan birinci, ikinci ve üçüncü derece hataların tamamının çözülmüş olması.

2.3 Hata sınıflandırması ve hata yaşam döngüsü

Grup hataları etkilerine göre üç dereceye ayırdı:

- **Birinci derece hata:** Sistemin genel çalışmasını doğrudan etkileyen hata.
- **İkinci derece hata:** Sistemin genel çalışmasını doğrudan etkilemeyen, ama işlevsel açıdan bazı özelliklerin çalışmasını engelleyen hata.
- **Üçüncü derece hata:** Sistemin çalışmasını etkilemeyen, görsel yönden ortaya çıkan hata.

Hata raporlamanın belirli bir prosedürü vardır. Test sonucunda bir hata bulununca önce bir hata bildirim formu düzenlenir. Hatayı bulan kişi formu doldurup hatayı sistem mühendisi grubuna iletir. Bu noktada hata yaşam döngüsünde (bug life cycle) “bildirildi” durumundadır. Sistem mühendisi grubu hatayı inceler. Kabul etmezse reddeder ve hata “kapandı” durumuna geçer. Kabul ederse hatayı ilgili kişiye atar. Atanan kişi hatanın kendisine ait olmadığını düşünürse hatayı tekrar sistem mühendisi grubuna gönderir. Kendisine aitse hatayı çözer ve test grubuna iletir; hata “testte” durumuna geçer. Test grubu gerekli testleri yapar ve hata giderilmişse “kapandı” olarak işaretler.

```
Hata bildirim formu doldurulur
  -> Sistem mühendisi grubuna iletilir            [Bildirildi]
       -> Reddedilirse                             [Kapandı]
       -> Kabul edilirse ilgili kişiye atanır      [Atandı]
            -> "Hata bana ait değil" -> sistem mühendisi grubuna geri döner
            -> Kişi hatayı çözer, test grubuna iletir   [Testte]
                 -> Testten geçerse                     [Kapandı]
```

2.4 Sorumluluklar, takvim, riskler ve ortam

Test sorumlulukları başlığında görevler ve tanımları verildi: konfigürasyon yönetimi sorumlusu, test yöneticisi, test grubu, test ortamı sorumlusu ve risk sorumlusu. Bu görevlere grup üyeleri atanmıştı. Test takvimi, üç testin hangi zaman aralığında yapılacağını gösteren bir Gantt şemasıyla verildi. Risk yönetiminde testlerin yapılmasıyla ilgili olası riskler ve bunlara karşı alınabilecek önlemler listelendi. Test ortamı olarak grup üyelerinin kendi kişisel bilgisayarları kullanıldı.

Bu kısımda hoca araya girip test planındaki görevlerin kişilerle eşleştirilmesini istedi: Hangi testi hangi üye yapmıştı? Planda risk sorumlusu görünüyordu ama birim testi ve sistem testi için kişi adı yazılmamıştı. Hoca bu eşleştirmenin plana eklenmesini ve sunum yapmayan üyelere bir sonraki derste projeyle ilgili soru soracağını söyledi.

> **Hocanın vurgusu:** Test planında kimin hangi testten sorumlu olduğu açıkça görünmelidir. Sorumluluk ataması planın bir parçasıdır.

2.5 Test sonuç raporu ve test verisi

Grup ardından test sonuç raporunun başlıklarını anlattı: test ortamı, test yaklaşımı, test durumlarının (test case) oluşturulması, test durumlarının izlenebilirliği (traceability), test prosedürü ve test durumu tanımları. Test durumu tanımları bölümünde test verisinin nasıl oluşturulduğu, test verisinin ne olduğu, test durumları ve bu durumların hangi gereksinimleri karşıladığı yer alır.

Test verisi elle (manuel) oluşturuldu. Veriler hazırlanırken potansiyel kullanıcıların girebileceği verilere benzer olmalarına dikkat edildi.

2.6 Kullanıcı testleri

Kullanıcı testleri için toplam altı test durumu belirlendi: beşi fonksiyonel gereksinimleri, biri fonksiyonel olmayan bir gereksinimi hedefliyordu. Grup bu testleri uygulama üzerinde canlı olarak gösterdi.

1. **Yöneticinin kullanıcı eklemesi ve düzenlemesi.** Yönetici rolüyle giriş yapılır, kullanıcı listesinden yeni kullanıcı eklenir. Kullanıcı eklerken ad, soyad, e-posta, şifre ve kullanıcı rolü girilmelidir. Kaydet tuşuna basıldıktan sonra “işlem başarıyla gerçekleştirildi” mesajı alınır. Yine de kontrol için kullanıcılar listesine gidilir ve yeni kullanıcının orada göründüğü doğrulanır.
2. **Yöneticinin kullanıcı araması.** Kullanıcı listesinde ad, soyad ya da e-posta yazılarak arama yapılır. Liste 50 sayfa da olsa, yazılan ada ya da soyada ait kullanıcılar gelmelidir. Ayrıntı simgesine tıklanınca kullanıcının ayrıntılı bilgisi görülür.
3. **Editörün kitap eklemesi ve düzenlemesi.** Editör rolüyle kitap ekleme sayfası açılır. ISBN, tarih, sayfa sayısı, yayıncı, yazar, kapak tipi ve kütüphanedeki konum gibi bilgiler girilir. Bir kısıt vardır: Yayıncı ve yazar yalnızca sistemde zaten var olanlar arasından seçilebilir; yeni yayıncı ya da yazar bu ekrandan tanımlanamaz. Kitap başarıyla eklendi mesajından sonra kitap listesinden de kontrol edilir.
4. **Kitap araması.** Kitap listesinde adın tamamı ya da bir kısmı yazılınca ilgili kitaplara erişilebilmelidir. Ayrıntı simgesiyle kitabın ayrıntılar sayfasına gidilir.
5. **Editörün yazar eklemesi ve düzenlemesi.** Yazar listesinden yeni yazar eklenir. Ekleme sonrası girilen bilgilere ait yazar profili ekrana gelirse işlem başarılıdır; yazarlar listesinden de kontrol edilir.
6. **Güvenlik (fonksiyonel olmayan gereksinim).** Bir kullanıcı yalnızca kendisine izin verilen modüllere erişebilmelidir. Root (yönetici) hesabı tüm modülleri görür. Kütüphaneci rolüyle girildiğinde yalnızca üye ekleme ve ödünç kayıtları modüllerine erişim vardır; kitap yalnızca aranabilir, kullanıcı ekleme, yazar ekleme ve kitap ekleme modüllerine erişim yoktur. Editör rolüyle girildiğinde kitap, yazar ve yayıncı yönetimi modüllerine erişim vardır, ama kullanıcı modülü görünmez; bu modül yalnızca yöneticiye aittir.

Altı testin hepsi başarılı sonuçlandı. Raporda ayrıca bir tablo ile her testin gereksinim analizindeki hangi fonksiyonel gereksinimi hedeflediği gösterildi. Bu, test durumları ile gereksinimler arasındaki izlenebilirliğin somut bir örneğidir.

> **Ders sonrası not:** Grup altıncı testi "authentication işlemi" olarak adlandırdı. Kimlik doğrulama (authentication) kullanıcının kim olduğunu doğrular. Bir rolün hangi modüllere erişebileceğini belirleyen işlem yetkilendirmedir (authorization). Bu testte sınanan şey yetkilendirmedir. Sunumu yapan da gösterim sırasında "yetkilendirme" sözcüğünü kullandı.

Bulunan hata. Test sırasında bir hata bulundu: Kitabı olmayan bir yazar eklendikten sonra kitaplar listesine erişilemiyordu. Grup bu hata için bir hata bildirim formu doldurup ilgili kişiye iletti. Böylece 2.3'te anlatılan prosedür gerçek bir hata üzerinde işletilmiş oldu.

2.7 Sistem testi: Web performans ölçümü

Sistem testi kapsamında performans testi yapıldı. Uygulama bir web uygulaması olduğu için web performans metrikleri kullanıldı. Ölçüm GTmetrix aracıyla yapıldı. Grup metrikleri şöyle açıkladı:

- **First Contentful Paint (FCP):** Sayfa yüklenmeye başladıktan sonra ekrana ilk içeriğin gelmesine kadar geçen süre.
- **Speed Index:** Arka planda yapılan bir içerik hesaplamasına dayanan, içeriğin ne hızla görünür hale geldiğini gösteren hız göstergesi.
- **Largest Contentful Paint (LCP)**, **Total Blocking Time (TBT)** ve **Cumulative Layout Shift (CLS)** gibi diğer metrikler. Grup bunların dersin ana konusu olmadığını belirterek yalnızca kısaca gösterdi.

Sonuçlarda toplam etkileşim süresi 0,8 saniye, sayfanın son kısmının yüklenmesine kadar geçen toplam süre 1,2 saniye olarak görüldü. Grup, bu sayıların tek başına iyi mi kötü mü olduğunu bilmenin zor olduğunu, aracın bunları genel web ortamıyla karşılaştırarak değerlendirdiğini söyledi. Yalnız CLS değeri 0,24 ile "kötü" değerlendirme aldı; diğer altı metrik iyi sonuç verdi.

> **Ek bilgi (derste anlatılmadı):** Google'ın Web Vitals ölçütlerinde CLS için büyük olasılıkla 0,1'e kadar "iyi", 0,1 ile 0,25 arası "iyileştirme gerekli", 0,25'in üstü "zayıf" sayılır. Buna göre 0,24 değeri sınırda bir "iyileştirme gerekli" sonucudur. Araçlar renklendirmeyi farklı eşiklerle yapabildiği için raporda kullanılan aracın kendi değerlendirmesi esas alınmalıdır. CLS, sayfa yüklenirken öğelerin beklenmedik biçimde kaymasını ölçer.

2.8 Birim testleri: Spring Boot Test ve JUnit

Proje bir Spring Boot projesi olduğu için birim testlerinde Spring Boot Test kütüphaneleri kullanıldı. Böylece testlerin üzerinde çalışacağı uygulama bağlamını (application context) oluşturmak ve testleri bu bağlama bağlamak gibi işler Spring'e bırakıldı. Testlerin kendisi JUnit ile yazıldı. Test kapsamındaki modüllerin servis katmanındaki kodlar test edildi.

Modül modül yapılan testler şunlardır:

- **Kitap modülü.** Yeni kitap ekleme ve kitabın ödünç almaya uygun olup olmadığını kontrol eden servis fonksiyonları test edildi. Kitap eklerken önce bir kitap kayıt formu oluşturulur ve servis kayıt fonksiyonuna parametre olarak verilir. Kayıt başarılıysa fonksiyon nesneyi, başarısızsa `null` döndürür; test bu dönüş değerini assert ederek kontrol eder. Uygunluk fonksiyonu `true` ya da `false` döndürür; bu da assert edilir.
- **Yayıncı modülü.** Kitap modülüne çok benzer. Yeni yayıncı kaydı için kayıt formu oluşturulur ve sonucun `null` ya da nesne olduğu assert edilir. Ayrıca ID'ye göre veritabanından yayıncı getiren fonksiyon test edildi: O ID'ye sahip bir yayıncı varsa nesne, yoksa `null` döner.
- **Kullanıcı modülü.** Kod üzerinden gösterildi. Yeni kullanıcı kaydında e-posta, ad ve şifre içeren bir kayıt formu nesnesi oluşturulur ve kullanıcı servisindeki kayıt fonksiyonuna verilir. Hata beklenmediği için sonuç `assertNotNull` ile değerlendirilir. Ek olarak girilen değerlerle kaydedilen değerlerin eşitliği de kontrol edildi.
  - Bu fonksiyon hatalı verilerde istisna (exception) fırlatır. Bunu sınamak için uygulama ayağa kalkarken kaydedilen root hesabının e-postasıyla yeni bir kayıt nesnesi oluşturuldu ve kayıt denendi. Testte "kayıt zaten var" türünden bir istisnanın fırlatılması gerektiği assert edildi.

  - Şifre girilmemişse de istisna fırlatılır. Şifresi `null` ya da boş dizgi olan bir kayıt nesnesiyle denenip "şifre boş" istisnasının fırlatıldığı assert edildi.
  - E-posta adresiyle veritabanından kullanıcı getiren fonksiyon da aynı biçimde test edildi.
- **Yazar modülü.** Yeni yazar kaydı yine sonucun `null` olup olmadığına göre assert edildi. Servisteki anahtar kelimeyle arama fonksiyonunun döndürdüğü sayfalı (page) sonuçtaki değerler beklenen değerlerle karşılaştırıldı.
- **Üye modülü.** Diğerlerine benzer biçimde kayıt ve ödünç almaya uygunluk fonksiyonları assert ile kontrol edildi.
- **Kütüphane ayarları modülü.** Bu ayarlar kütüphanenin iç işleyişini kontrol eder. Örneğin `maxExtension` ayarı bir ödünç kaydının kaç kez uzatılabileceğini belirler. Önce bu isimle servisten kayıt istendi: Bulunursa `assertNotNull`, bulunamazsa `null` beklenir. Ardından düzenleme test edildi: `maxExtension` değeri 5 yapıldı ve sonuç assert ile kontrol edildi. Güncelleme fonksiyonu hatalı durumlarda istisna fırlatır: `maxExtension` sayısal değer kabul ettiği için ona metin tipinde bir değer verildiğinde istisna fırlatıldığı test edildi.

Bu testlerde iki tür durumun birlikte sınandığına dikkat etmek gerekir: geçerli girdide beklenen sonucun üretilmesi (pozitif test) ve geçersiz girdide sistemin doğru biçimde hata vermesi (negatif test). Var olan e-postayla kayıt, boş şifre ve sayısal alana metin verilmesi negatif test örnekleridir.

3. Hocanın Birinci Gruba Değerlendirmesi ve Projede Beklenen Minimum Koşullar

Sunum bittikten sonra hoca grubun dört üyesini sunumdaki bölümlerle eşleştirdi: kimin planlamayı, kimin kullanıcı ve sistem testlerini, kimin birim testini anlattığını teyit etti. Sunum yapmayan üyeye bir sonraki hafta sunumun herhangi bir yerinden soru soracağını söyledi.

Hoca sunumu başarılı buldu. Bu hafta az sunum olduğu için birinci gruba fazla zaman verdiğini, bir sonraki hafta sunum yapacak grupların daha planlı ve daha özet anlatması gerektiğini belirtti. Sınıfın kalabalık olduğunu, ilk grubun sunumunun diğerleri için aydınlatıcı olduğunu düşündüğünü söyledi.

Hoca bu grubun, projede istediği minimum koşulları yerine getirdiğini vurguladı. Bu koşullar şunlardır:

- **Yazılımın genel tanıtımı.** İsteyen grup gereksinim (SRS) raporunu ortaya koyarak yazılımı onun üzerinden anlatabilir.
- **Test planı.**
- **Kullanıcı testi ve sistem testi için en az bir açık kaynaklı (open source) araç.** Grupların böyle bir araç bulup kullanması istenmişti.
- **Birim testi.** Kod üzerinden anlatılmalıdır.

> **Hocanın vurgusu:** Projede beklenen minimum koşullar yazılımın tanıtımı, test planı, kullanıcı ve sistem testi için en az bir açık kaynaklı araç ve kod üzerinden anlatılan birim testidir. Birinci grup bu koşulları yerine getiren bir örnektir.

4. İkinci Grup: Mikroservis Mimarili Dil Eşleştirme Platformu

4.1 Yazılımın tanıtımı ve mimarisi

İkinci grubun projesi, grup üyelerinin yaz aylarında birlikte geliştirdiği bir sosyal medya platformudur. Kullanıcı öğrenmek istediği dili belirler; sistem onu belirli kriterlere göre o dili bilen kişilerle eşleştirir. Hoca projenin bir yerde kullanılmak üzere yapılıp yapılmadığını, bir müşterisi olup olmadığını sordu. Grup bir müşterileri olmadığını, sistemi kendi oluşturdukları kullanıcılarla test ettiklerini söyledi. Hoca ayrıca grup üyelerinin adlarının bulunduğu slayta dönülmesini istedi; grup her servisin testini hangi üyenin yaptığını bu slaytta gösterdi.

Sistem mikroservis (microservice) mimarisiyle tasarlanmıştır ve birçok ayrı servisten oluşur:

Servis	Teknoloji	Görev
User Info Service	Node.js, Express.js	Kullanıcı bilgileri; işin çoğu bu servisten yürür, en büyük servistir
Authentication Service	Node.js, MongoDB, Mongoose, bcrypt, JWT	Giriş ve yetki verme
Message Service	Node.js, Express.js	Eşleşen kullanıcılar arasında mesajlaşma
Match Service	Python, FastAPI	Kullanıcılara arkadaş önerme
Web Client	Vue.js, Vuetify; Heroku üzerinde çalışır	Ön yüz

Test araçları servislerin teknolojisine göre seçildi:

- Node.js servislerinde birim testleri Mocha ve Chai ile yazıldı, kapsam (coverage) Istanbul ile ölçüldü.
- Entegrasyon testlerinde Sinon ve SuperTest kullanıldı.
- Sistem testleri Postman ile yapıldı.
- Match Service'in birim ve entegrasyon testleri PyTest ile yapıldı.
- Web Client'ın birim testleri Vue Test Utils ile, sistem testleri Chrome Lighthouse ile yapıldı.

  Ek bilgi (derste anlatılmadı): Mocha bir test çalıştırıcısıdır (test runner); `describe` ve `it` bloklarıyla test yazılmasını sağlar. Chai bu testlerdeki doğrulama (assertion) ifadelerini sağlar. Sinon, bir fonksiyonun yerine geçen sahte nesneler (stub, mock, spy) üretir; böylece test edilen birim, bağımlı olduğu veritabanı ya da başka servislerden yalıtılabilir. SuperTest ise bir Express uygulamasına gerçek HTTP istekleri gönderip yanıtı doğrulamak için kullanılır.

4.2 Gereksinimlerden test raporuna

Grup önce gereksinim spesifikasyonu belgesine değindi. Belgenin fonksiyonel gereksinimler kısmında kullanım senaryoları, 3.2 bölümünde ise bütün harici arayüz gereksinimleri tanımlanmıştı. Yazılım test raporunda birim, entegrasyon ve sistem testlerindeki bütün test edilen öğeler, SRS'teki ilgili başlıklarla eşleştirildi. Böylece her testin hangi gereksinime karşılık geldiği izlenebilir hale geldi.

Raporda çok sayıda test bulunduğu için yalnızca bir örnek anlatıldı. Test grupları (test suite) genellikle servislerin adlarıyla eşleşir. Test edilen birimler çoğunlukla arka uç servislerindeki controller, model ve validation birimleridir. Örnek test şöyleydi:

- **Test başlığı:** Kullanıcıların doğrulanması.
- **Test senaryosu:** Boş kullanıcı listesinin gelmesi.
- **Girdi:** Yok. Test yalnızca istek gönderilen fonksiyonun boş kullanıcı listesini doğru biçimde döndürüp döndürmediğini kontrol eder.
- **Beklenen sonuç:** Boş liste. Normalde listede kullanıcılar olması beklenir; bu test hiç kullanıcı yokken sistemin nasıl yanıt verdiğini sınar.
- **Sonuç:** Test geçti, hata bulunmadı.

Bu örnek, sınır durumunun (hiç kayıt olmaması) ayrı bir test durumu olarak ele alınmasının tipik bir örneğidir.

4.3 User Info Service: birim, entegrasyon ve kapsam

Grup User Info Service'in testlerini kod üzerinde çalıştırarak gösterdi. User controller içinde çok sayıda test durumu vardı. Birim testleri çalıştırıldığında bu servis için toplam 49 birim testi yazıldığı ve hepsinin geçtiği görüldü.

Entegrasyon testlerinde Sinon ve SuperTest kullanıldı. Grup burada bir sorundan söz etti: Node.js'teki bir hata yüzünden bazı durumlarda bir önbellek (cache) silme problemi oluşabili-

yordu. Testler bulutta (cloud) çalıştırıldığında sorun yoktu, ama yerelde (local) çalıştırıldığında Node.js ortamından kaynaklanan hatalar görülebiliyordu.

> **Ders sonrası not:** Ortama göre sonucu değişen bir test güvenilir değildir. Bu tür testlere kararsız test (flaky test) denir. Testin yerelde başarısız olup bulutta geçmesi kabul edilmek yerine sebebi bulunup giderilmeli ya da raporda açık bir risk olarak yazılmalıdır.

Kapsam ölçümünde Istanbul kullanıldı. Araç bütün testleri çalıştırdıktan sonra modül modül kapsam değerlerini bir klasöre raporlar. Grubun test planındaki geçme ve kalma kriterine göre sistemlerde %60 test kapsamına ulaşılması yeterli görülmüştü. User Info Service'te ölçülen kapsam %64'tür. Bu değerin dağılımı şöyledir: validation ve models kısımlarında %100, controllers kısmında %63.

> **Ders sonrası not:** %60 kapsam hedefi grubun kendi incelemesiyle seçtiği bir değerdir; evrensel bir eşik yoktur. Kapsam hedefi, projenin riskine ve kodun türüne göre test planında gerekçesiyle yazılmalıdır. Ayrıca toplam %64 değeri büyük olasılıkla modüllerin satır sayılarına göre ağırlıklı bir ortalamadır; altyazıda satır sayıları verilmediği için bu dağılım doğrulanamadı.
>
> **Ek bilgi (derste anlatılmadı):** Kapsam ölçümünün farklı türleri vardır. Satır ya da deyim kapsamı (statement coverage) çalıştırılan satırların oranını, dal kapsamı (branch coverage) `if` gibi karar noktalarındaki her dalın çalıştırılıp çalıştırılmadığını, fonksiyon kapsamı ise çağrılan fonksiyonların oranını ölçer. Istanbul gibi araçlar bu oranları ayrı ayrı raporlar. Yüksek kapsam, testlerin doğru şeyi doğruladığını tek başına garanti etmez.

Sistem testi. Sistemin yükünü ve yanıtını test etmek için Postman kullanıldı. Postman'de URL'lere çeşitli HTTP istekleri gönderilebilir, başlık (header) ve gövde (body) eklenebilir, testler yazılabilir ve bu testler test runner ile toplu çalıştırılabilir. Grup canlı (production) ortamdaki User Info Service'e bağlanıp fonksiyonların canlı ortamda doğru çalışıp çalışmadığını sınadı. Toplam 9 sistem testi vardı ve hepsi geçti. Her testte yanıt süresinin (response time) 200 milisaniyenin altında olması kontrol edildi.

4.4 Test araçları nasıl seçildi?

Hoca bu noktada, bir sonraki hafta sunum yapacak gruplar için de yararlı olacağını belirterek grubun araçları nasıl araştırıp seçtiğini sordu. Grubun cevabı şöyle özetlenebilir:

- Uygulamanın hangi dil ve platformlarda yazılacağı önceden belliydi. Bu, aday araçları daralttı.
- Node.js kullanıldığı için Node.js'in paket yöneticisi npm'in sitesinde kütüphaneler kategorilerine göre tarandı. Bazı kütüphaneler şifreleme, bazıları test içindir; test kategorisindeki kütüphaneler incelendi.
- Adayların kendi siteleri ve Medium yazıları incelendi. Seçim Mocha ile Jest arasında kaldı. Karar verirken daha çok ve daha kaliteli yazısı olan, sisteme daha uygun ve test edilebilirliği yüksek kod örnekleri içeren kaynaklara bakıldı. Sonunda arka uç için Mocha seçildi.
- Web Client Vue.js ile yazıldığı için Vue.js'in doğrudan belgelendirdiği ve desteklediği Vue Test Utils kullanıldı. Bu araç Jest altyapısını kullanır. Mocha ile Jest temelde birbirine çok benzer, yazım stilleri de benzerdir. Yani arka uçta Mocha, ön yüzde Jest tabanlı bir araç kullanıldı.

Bu cevap, araç seçiminde izlenebilecek bir yöntemi gösterir: önce platformun belirlediği adayları bulmak, sonra belgelendirme, topluluk kaynakları ve projeye uygunluk açısından karşılaştırmak.

4.5 Authentication Service

Bu bölüm hocanın internet bağlantısı koptuğu için daha sonra özet olarak tekrar anlatıldı. Authentication Service, web istemcisini veritabanına bağlayan ve kullanıcı bilgilerini girip gerçekten o kullanıcı olduğunu kanıtlayan kişiye işlem yapma izni veren servistir. Veritabanı olarak MongoDB, MongoDB'ye erişmek için Mongoose kütüphanesi kullanıldı.

Şifreler bcrypt kütüphanesiyle özetlenerek (hash) veritabanına kaydedilir. Grup burada karşılaştığı bir sorunu anlattı: Kullanıcılar genellikle basit şifreleri tekrar tekrar kullanır. Örneğin 1'den 8'e kadar rakamların sıralanmasından oluşan şifreler için üretilen özetler aynı olur ve bu bir güvenlik açığıdır. Grup bu sorunu aynı kütüphanedeki tuzlama (salting) işlemiyle çözdü.

> **Ders sonrası not:** Grup işlemi "önce özetleme, sonra tuzlama" diye anlattı. Doğrusu: bcrypt her şifre için rastgele bir tuz (salt) üretir ve şifreyi bu tuzla birlikte özetler; tuz, sonuçtaki özet dizgisinin içinde saklanır. Böylece aynı şifreler farklı özetler üretir ve önceden hesaplanmış özet tablolarıyla (rainbow table) yapılan saldırılar zorlaşır.

Giriş başarılı olunca kullanıcıya bir JSON Web Token (JWT) döndürülür. Kullanıcı sonraki işlemlerde e-posta ve şifre yerine bu token ile işlem yapar. Grup token'ı "bir nevi cookie" diye anlattı ve token'ın cookie'lerden farklı olarak tarayıcının yerel deposunda (local storage) tutulduğunu söyledi. Bu servisin birim testleri Mocha, Chai ve Istanbul ile, entegrasyon testleri Sinon ve SuperTest ile, sistem testleri Postman ile yapıldı.

> **Ders sonrası not:** JWT bir cookie türü değil, bir token biçimidir; cookie içinde de local storage'da da saklanabilir. Local storage JavaScript'ten okunabildiği için XSS (cross-site scripting) saldırısına açıktır; bu yüzden büyük olasılıkla HttpOnly cookie sık önerilen bir seçenektir.

4.6 Message Service ve Match Service

Message Service. Node.js ve Express.js ile yazıldı. Match Service ile eşleşen kullanıcıların birbiriyle mesajlaşmasını sağlar. Çalışma biçimi şöyledir: Bir kullanıcı diğerine ilk kez mesaj attığında veritabanında bu iki kullanıcı adından bir sohbet odası (chat room) oluşur. Mesajlar orada tutulur. Kullanıcılar sonraki girişlerinde mesajlara buradan çekilerek ulaşır. Birim testleri Mocha, Chai ve Istanbul ile, entegrasyon testleri Sinon ve SuperTest ile, sistem testleri Postman ile yapıldı. Sohbet için iki birim testi yazıldı.

Match Service. Kullanıcılara arkadaş önermek için Python ve FastAPI ile yazıldı. Servis, User Info Service'ten kullanıcıların hobilerini, dillerini, ülkelerini ve okullarını alır. Bu değerler bir metne (string) çevrilip vektörleştirilir ve kosinüs benzerliği (cosine similarity) hesaplayan fonksiyona verilir. Fonksiyon en benzer 30 kullanıcıyı döndürür. Birim ve entegrasyon testleri PyTest ile yapıldı.

4.7 Web Client, sürekli entegrasyon ve kullanıcı testi

Web Client. Vue.js ile yazılmış ön yüzdür; kullanıcının bütün arka uç servisleriyle iletişim kurmasını sağlar ve Heroku üzerinde çalışır. Grup uygulamayı canlı olarak gösterdi: Giriş sayfasında Authentication Service çalışır, buradan kayıt ya da giriş yapılır. Ana sayfada Match Service'in önerdiği kullanıcılar listelenir. Bir kullanıcının profiline gidildiğinde bilgiler User Info Service'ten alınır; kişinin özellikleri, hobileri, bildiği ve konuşmak istediği diller görülür. Mesajlar kısmında son konuşulan kişiler ve mesajlaşmalar, ayarlar kısmında hesap ayarları vardır.

Web Client'ın sistem testi Chrome Lighthouse ile yapıldı. Uygulama Heroku'nun ücretsiz sunucusunda çalıştığı için düşük performans bekleniyordu, ama performans puanı 92 çıktı. Erişilebilirlik (accessibility) konusunda Vue.js ve Vuetify'ın sağladığı özellikler kullanıldığı ve en iyi uygulamalar (best practices) izlendiği için Lighthouse puanları yüksek çıktı. Raporda testlerin hangi sistem üzerinde yapıldığına dair bilgiler de yer aldı.

Sürekli entegrasyon (continuous integration). User Info Service'in GitHub deposunda bir GitHub Actions YAML dosyası vardır. Master dalına her push ve pull request işleminde bu iş akışı tetiklenir: Node.js ortamında önce derleme (build) yapılır, sonra birim testleri çalıştırılır. Grup ayrıca test planının 18. bölümündeki onaylamalar kısmına ve Istanbul kapsam raporunun ekran görüntüsüne değindi.

> **Ek bilgi (derste anlatılmadı):** Sürekli entegrasyonda her kod değişikliği ortak depoya gönderildiğinde otomatik derleme ve test çalıştırılır. Amaç, hatanın değişikliği yapan kişi henüz o kodu hatırlarken yakalanmasıdır. Bu yapı birim testlerini bir ke-

relik bir etkinlik olmaktan çıkarıp sürekli çalışan bir regresyon testi (regression test) güvencesine dönüştürür.

Kullanıcı testi. Uygulama alfa aşamasındadır ve herkese açıktır. Yaklaşık 20-25 kullanıcı uygulamayı aktif ve yoğun biçimde kullanır. Bu kullanıcılardan masaüstü ve mobil cihazlarda, farklı web tarayıcılarında ve farklı işletim sistemlerinde geri bildirim alındı. Böylece kullanıcı testi farklı ortam kombinasyonlarını kapsadı.

5. Üçüncü Grup: Otobüs Bilet ve Yolculuk Yönetim Sistemi

5.1 Yazılımın tanıtımı

Üçüncü grup iki kişiydi. Seçtikleri yazılım otobüs biletleri ve yolculukları yönetmek için tasarlanmış bir sistemdir; bir önceki yıl başka bir dersin projesi olarak geliştirilmişti. Hesabı olmayan kullanıcı kayıt olarak giriş yapabilir. Yönetici olarak girildiğinde otobüsler görüntülenir, yeni otobüs eklenir, güncellenir ya da silinir. Çalışanlar ve yöneticiler görüntülenip düzenlenebilir. Yolculuklar kısmında seferin nereden kalkıp hangi şehre gittiği görülür. Kullanıcılar ve biletleri de görüntülenebilir. Sistem bilet almak üzerine tasarlanmıştır: Her kullanıcının bir bakiyesi vardır ve kullanıcı sisteme girerek bilet alır.

5.2 Test planı ve senaryolar

Grup birim testinde nasıl ilerleyeceklerini birlikte kararlaştırdığını ve sıralama şemalarıyla (sequence diagram) gösterdiğini anlattı. Planlanan testler şunlardır:

- **Birim testi:** İki senaryo incelenir. Birincisinde kullanıcı adı ya da şifre eksik verilerek giriş denenir. İkincisinde yanlış bilgilerle giriş denenir.
- **Sistem testi:** Sistemin veritabanıyla gerçekten bağlantı kurup kuramadığı ve doğru hata kodlarını döndürüp döndürmediği incelenir.
- **İstemci testi:** İstemciden (client) bir giriş izni istenir ve bu iznin geri dönüp dönmediği kontrol edilir.

Zaman planı bir Gantt şemasıyla verildi: ilk kısım SRS belgesinden önceki çalışmayı, sonraki kısım test aşamalarını gösteriyordu.

Öncelikli senaryolar iki taneydi. Birincisi sisteme yeni müşteri eklenmesidir; grup bunun sistemin en önemli noktalarından biri olduğunu, eklemenin doğru çalıştığını ve hata durumunda hatanın gerçekten döndürüldüğünü kontrol ettiğini söyledi. İkincisi sisteme giriştir. Bu da kritiktir, çünkü yetkisiz bir kişi sisteme girebilirse kendisine ait olmayan bilgileri değiştirebilir ya da başkası adına bilet alabilir.

Bu gerekçelendirme, test önceliğinin riske göre belirlenmesine bir örnektir: En çok zarar doğurabilecek işlevler (kimlik doğrulama, kayıt) önce ve daha ayrıntılı test edilir.

5.3 Entegrasyon ve birim testlerinin çalıştırılması

Sunumun ikinci yarısında ses çok bozuk geldi ve altyazının büyük kısmı anlaşılmaz. Çıkarılabilen bilgiler şunlardır:

- Entegrasyon testi için sahte (dummy) test verileri içeren ayrı bir test veritabanı ayağa kaldırıldı.
- JUnit testleri, önceden belirlenen kullanıcı bilgileriyle REST istekleri göndererek otomatik olarak kayıt olmayı ve giriş yapmayı deniyordu.
- Canlı gösterimde iki birim testi çalıştırıldı ve grup bunların başarıyla geçtiğini söyledi. Testlerin ikisi de giriş işlemiyle ilgiliydi; planda bunlar eksik bilgiyle ve yanlış bilgiyle giriş senaryoları olarak tanımlanmıştı.
- Entegrasyon testleri ayrı bir komutla çalıştırılabiliyordu.

Hoca sesin çok az geldiğini, ama raporla birlikte takip edebildiğini söyledi. Grubun iki kişi olduğunu hatırlatarak zaman planlarını daha iyi yapmaları gerektiğini belirtti.

6. Son Hafta Sunumları İçin Yönlendirmeler

Hoca dinleyen gruplara, bir sonraki haftaki sunumlarıyla ilgili soruları olup olmadığını sordu. Bir sonraki ders dönemin son dersiydi ve kalan bütün grupların sunum yapması gerekiyordu. Hocanın sunum içeriğine dair yönlendirmeleri şunlardır:

- Uygulama kısaca tanıtılmalıdır.
- Birim testi anlatılırken kodlar kullanılmalıdır.
- Slayt hazırlamak önerilir. Raporun tamamını baştan sona göstermek yerine ana hatları bir sunum halinde birleştirmek sunumları hızlandırır.
- Her sunum en fazla 20 dakika sürmelidir.

Ders sonunda bir sonraki haftanın ilk üç grubunun sırası belirlendi. Kalan grupların sırasını hoca derste yapacaktı. İlk üç gruptan sonra bütün grupların derste bulunması istendi.

Örnek Sorular

Bu derste ekran karesi yoktur. Aşağıdaki sorular hocanın sunumlar sırasında gruplara sözlü olarak sorduğu sorulardır.

Soru 1 (derste sözlü, 00:16:28): Test planında hangi test hangi grup üyesi tarafından yapıldı? Birim testi ve sistem testinin sorumluları planda görünüyor mu?

> **Cevap (derste verildi):** Planda risk sorumlusu yazılıydı, ama birim ve sistem testlerinin sorumluları görünmüyordu. Grup birim testini yapan üyeyi sözlü olarak söyledi. Hoca bu eşleştirmenin plana eklenmesini istedi. Ders bağlamındaki çıkarım şudur: Test planının “sorumluluklar” bölümünde her test faaliyeti bir kişiye atanmalıdır.

Soru 2 (derste sözlü, 00:38:57): Bu yazılımı bir yerde kullanılmak üzere mi yaptınız, bir müşterisi var mıydı?

> **Cevap (derste verildi):** Hayır. Proje yaz aylarında grup tarafından geliştirildi; müşterisi yoktu. Sistem, grubun kendi oluşturduğu kullanıcılarla test edildi. Sonradan uygulama alfa aşamasında herkese açıldı ve yaklaşık 20-25 aktif kullanıcıdan geri bildirim alınarak kullanıcı testi yapıldı.

Soru 3 (derste sözlü, 00:45:51): Kullandığınız test araçlarının seçimini nasıl araştırdınız?

> **Cevap (derste verildi):** Yazılımın dili ve platformu önceden belliydi. Node.js kullanıldığı için npm sitesinde test kategorisindeki kütüphaneler tarandı. Adayların siteleri ve Medium yazıları incelendi; seçim Mocha ile Jest arasında kaldı ve arka uç için Mocha seçildi. Ön yüz Vue.js ile yazıldığı için Vue.js’in resmî olarak desteklediği, Jest altyapısını kullanan Vue Test Utils tercih edildi. İki aracın yazım stili birbirine çok benzer.

Dersin Toparlaması

Bu derste üç grup, dönem boyunca teorik olarak görülen test planı ve test sonuç raporu yapısını kendi projelerine uyguladı. Birinci grup tek parça bir Spring Boot web uygulamasında üç seviyeli bir test planı, üç dereceli hata sınıflandırması, bir hata yaşam döngüsü, gereksinimlere izlenebilen altı kullanıcı testi, GTmetrix ile performans ölçümü ve JUnit ile servis katmanında pozitif ve negatif birim testleri gösterdi. İkinci grup mikroservis mimarisinde her servise teknolojisine uygun araç seçmenin, kapsam hedefi koymanın, Postman ile canlı ortamda yanıt süresi testinin ve GitHub Actions ile sürekli entegrasyonun nasıl yapılacağını gösterdi. Üçüncü grup ise risk temelli önceliklendirmeyle giriş ve kayıt senaryolarını, ayrı bir test veritabanı üzerinde JUnit ile entegrasyon testlerini anlattı.

Hocanın değerlendirmesinden çıkan ana dersler şunlardır: Projede yazılımın tanıtımı, test planı, kullanıcı ve sistem testi için en az bir açık kaynaklı araç ve kod üzerinden birim testi bulunmalıdır. Test planında her faaliyetin sorumlusu açıkça yazılmalıdır. Araç seçimi gerekçelendirilmelidir. Sunumlar zaman planına uyacak biçimde özetlenmelidir. Bir sonraki ders, kalan grupların aynı yapıdaki sunumlarıyla dönemi tamamlayacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.